D-NPGA : a new approach for tasks offloading in fog/cloud environment

1st Léo Bernard
CY Cergy Paris University
Cergy, France
leo.denis.bernard@gmail.com

2nd Sonia Yassa
ETIS UMR8051 CNRS CY Cergy Paris
University, Cergy France
sonia.yassa@cyu.fr

3rd Lylia Alouache
ETIS UMR8051 CNRS CY Cergy Paris
University, Cergy France
lylia.alouache@cyu.fr

***Abstract*—In recent years, the Internet of Things (IoT) has gained significant attention as a platform for various smart applications that rely on sensors to collect and process data. However, the transmission of data from IoT devices to Cloud data centers can be delayed due to the physical distance between them. Fog computing has emerged as a promising solution to address this issue by bringing the computing resources closer to the IoT devices. One of the challenging problems in Cloud/Fog environment is tasks offloading. A lot of computer work is required to optimize even a small number of tasks. In this paper, we propose a new algorithm based on the Niched Pareto Genetic Algorithm (NPGA) called D-NPGA (Drafting NPGA) to tackle this problem. Our results show that this method can outperform state of the art solution and optimize the total cost and the makespan by 15%. This algorithm also scales better than its counterparts gaining an important advantage as the number of tasks rises. Besides, it proposes a set of optimal solutions instead of a single one.**

***Index Terms*—Tasks offloading, Cloud, Fog, NPGA**

I. Introduction

A. Background and motivation

During the last 20 years more and more objects have been connected through what is called the Internet of Things (IoT). The subjects have varied nature such as sensors, smart house equipment or connected watch. The number of these objects have grown to 14.3 billion of them in 2022 [3] and keeps getting bigger. The reason for this increase is the large number of situation it can be used. In the industry, machines and processes can be monitored by IoT sensors to detect defects and prevent breakdowns. Farms may also utilize these to optimize certain tasks. It can also be used for more day to day life use cases. For example, smart houses are filled with sensors that control temperature, turn on and off the light and keep track of the content of the fridge. One of the domain that benefited the most from IoT is vehicle. With the development of connected devices, it has been possible to make fully autonomous cars that gather data from their surrounding and exchange information with other cars to safely drive. In general, IoT has developed at a constant rate the last few years. It is estimated that 73 zettabytes of data will be exchange via IoT in 2025 [1]. Analysing that much data requires enormous computation power that IoT devices don't have. Therefore, something else must bee used. Traditionally, this has been the role of cloud computing. By allowing users to work with powerful computers connected to the network, it has been possible to offload IoT tasks on these machine and reduce computation time and cost by a large margin. Computing intensive tasks can be executed on computers with high memory, cpu power and stored in large database. But, one issue with this architecture is latency. Indeed, data centers used for cloud computing are generally far from end users. This can increase the cost and latency of the operation because of bandwidth usage. This is why in 2015, Fog computing was created with the idea to bring closer computation power and end users. It is made of many devices placed all around the user with limited computing capabilities [2]. Therefore, small tasks can be directly executed on fog node while big tasks are split into tiny piece, sent to fog and cloud node and brought back together.

B. Goals

As we said, cloud and fog environment are very good at managing computation intensive task. But, a key problem that arise when doing so is that nodes have different characteristics. Firstly, cloud and fog nodes have opposite strength and weaknesses, cloud machines being powerful but costly and fog nodes being cheap and close to user but having poor computation power. Secondly, even machine in the same environment have different price, cpu power or energy consumption. Therefore, one repartition of task won't take the same time or cost the same amount as another. The task offloading problem seeks to optimize this repartition in order to maximize or minimize certain Quality of Service (QoS). This is one of the major problem to solve in cloud computing and is wildly studied in the literature [], [], []. We choose to tackle this issue by proposing a new genetic algorithm called D-NPGA. A genetic algorithm generate a population of individual and makes it explore the space to find optimized solutions. To do so, it uses a selection step to select the best individuals and crossover to generate new solutions based on the selected ones.

In this work, we propose a novel solution based on NPGA [7] to offload IoT tasks on the appropriate fog/cloud nodes. Our contribution is resumed in the next points :

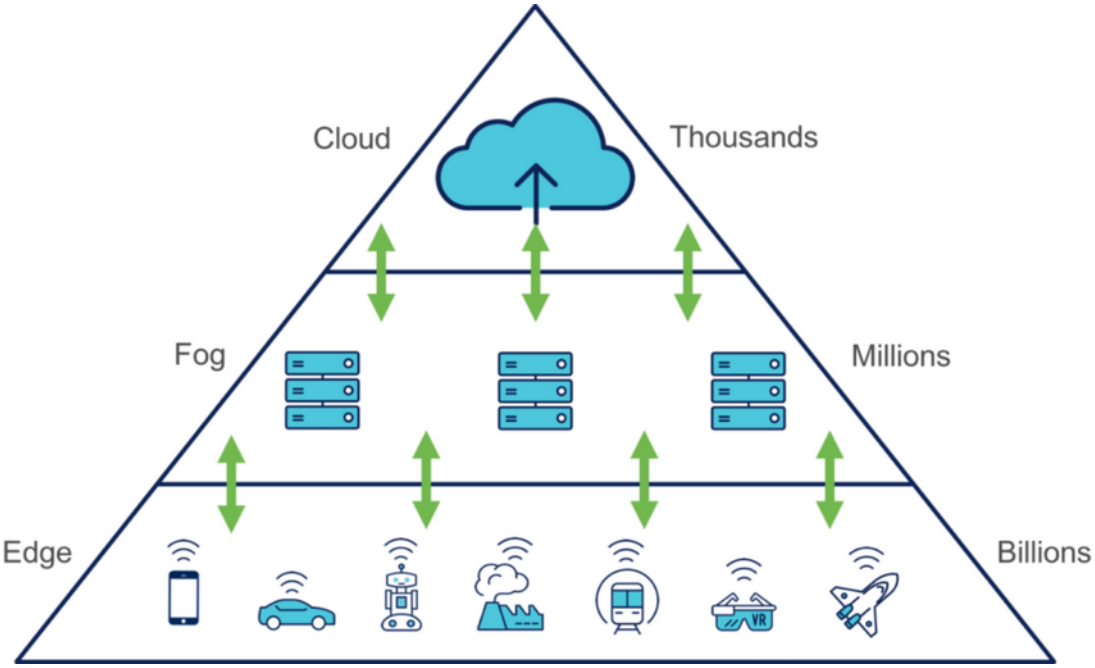

Fig. 1: Schema of Cloud/Fog architecture

- We adapt a genetic algorithm to tackle the task offloading problem while considering different metrics
- We use the multi-objective nature of metaheuristics to generate a pareto front.
- We enhanced NPGA metaheursitic by adding a drafting step to enhance the number of pareto solution produced

C. Paper organization

The rest of this paper is organized as follows. Section II presents the related work on tasks offloading and scheduling in a Fog/Cloud context. In Section III, we explain the mathematical background and our problem formulation. The details of our proposed D-NPGA approach are presented in Section IV. In Section V, we present the experimental study, the obtained results and their comparison to some state of the art algorithms. Section VI, concludes this work.

II. Related Work

Tasks offloading in a Fog/Cloud environment is a well studied problem in the scientific literature. As the environment is very complex, a number of different approaches were developed to tackle specific use cases.

The most direct way to solve tasks offloading problems is to use a deterministic algorithm. In their paper, J.Tsai et al. [17] uses a deterministic method to optimise both the makespan and the cost of tasks offloading. This method uses linear transformation techniques to solve the problem and optimzes the qualities of service (QoS). An important feature of deterministic method is their execution speed. L. Liu et al. focuses on this aspect in their article [18]. They propose an algorithm to optimize the cost and execution time of task offloading in Cloud-Edge environment. Their work seeks to improve the speed of state of the art algorithms without degrading performances. Researches also focused on the security of these methods. Razaque et al. presented an energy efficient and secure hybrid (EESH) algorithm to prevent malicious attack and optimize energy consumption at the same time. While the results are very good, the scaling and resource usage of these algorithms make them difficult to use in complex environment.

Another direction is to use a heuristic to find an approximation of the optimal solution to the problem. One such approach is to use a priority queue, as demonstrated by Adhikari et al. [5]. In their paper, they use it to tackle the tasks offloading problem with delay constraint. The proposed algorithm outperformed other methods in terms of makespan while reducing the starvation for low priority tasks. A multi-criteria approach was proposed by Choudhari et al. [6]. This solution aims to optimize the makespan and cost of the final solution in fog and cloud environments. The experiments conducted by the authors demonstrate that it efficiently reduces these two metrics. Fuzzy logic is another explored method to tackle task offloading. In [], Almutairi and Aldossary presented a Fuzzy logical algorithm. They sought to minimize latency and maximize ressource efficiency for IoT tasks in Cloud and edge environment. Although they scale better than deterministic methods, these methods generally produce worse solution in term of QoS than other approachs.

Metaheuristics have often been used for this type of problem, as they are good at tackling complex tasks with multiple

metrics to optimize. Hoseiny et al. [8] created a priority aware genetic algorithm (PGA) to generate a solution for a cloud/fog environment. They aimed to reduce latency and showed that this would also reduce energy consumption. Indeed, the proposed solution optimised these metrics better than the comparison algorithm. Genetic algorithm are often hybridized with other method to get better resutls. In their paper, Wang et al. presented a hybrid genetic algorithm with integer coding that. it maximized the ressource efficiency, user satisfaction and processing efficient for an E3C environment. Another very popular metaheuristic is particle swarm optimization (PSO). Adhilari et al. used it in [] to offload IoT tasks in a Fog computing framework. They created a solution called DPTO that was able to minimize cost and makespan better than compared algorithms. A key aspect of metaheuristic is that they are very good for multi objective optimization. They can produce many optmized solutions at the same time and therefore explore different tradeoff while having the same execution time as normal metaheuristics. This can be seen in [11]. In it, Luo et al. use a PSOCO algorithm to optimise the delay and cost of tasks offloading for an edge framework. Their solution generates Pareto optimal solutions. Experiments showed that it improved performance compared to traditional metaheuristics.
Besides these two well-known algorithms, there are many more algorithms inspired by natural phenomena such as marine predators [4] or invasive weeds [9]. Metaheuristics can find a good solution in a reasonable time in a complex environment, but they can be too slow for real-time decision making.

Recently, many good results have been obtained in scientific papers using machine learning approaches. The capacity to learn from previous experience and quick execution time after training have proven to be very useful for this type of problem. Deep reinforcement learning (DRL) has been used by Li et al. [10] in an edge environment. Their goal was to obtain good solutions that minimise energy consumption while having the ability to drop certain task if necessary. Combined with K-NN, the solution outperformed the comparison algorithm on all metrics and was also able to perform well under a delay constraint. DRL can also be combined with other algorithm to enhances performances even further. For example, it was used with Markov chains in the work of Ning et al. [13]. They approached the performance of the deterministic method while being significantly faster. DRL was also used with LSTM by Prathiba et al. []. Their algorithm optimized computation rate better than other DRL approachs in a MEC environment. Another possible approach is to use Federated learning as shown by Prathiba et al. In their paper [], they proposed a FLOR algorithm to tackle task offloading and ressource management in 6G-V2X environment. The proposed solution showed better result in optimzing cost, delay and latency than compared methods. A recent approach in Deep neural network is meta learning. This method uses a meta-idea algorithm that is trained to select the meta parameters of the main algorithm in order to have better performances. Gali et al. used it in their paper [] and created DDMTO to tackle task offloading in smart cities. While the performances of these algorithms are undeniable, they still have drawbacks. Firstly, a lot of data needs to be collected in order to train the model correctly. This can be difficult to find due to confidentiality issues. Also, machine learning solutions can't generate a list of Pareto optimal solutions like metaheuristics, which can be interesting when QoS preferences change.

Based on our analysis of related work methods presented in this paper, we find that metaheuristics offer a balanced approach with good performance, reasonable makespan, memory consumption, and ease of implementation. However, it is worth noting that most proposed methods use aggregation to transform the problem into a single objective optimization, which may limit flexibility for the user. Methods that use multi-objective optimization often face scalability issues, as demonstrated in this review [16]. Therefore, we addressed these problems by implementing a novel solution D-NPGA in a Fog/Cloud environment to optimize both the cost and makespan of a given set of tasks. By doing so, we optimized the state of the art performances in both QoS with a better scaling than the other proposed solutions.

III. Problem modelling

In this paper we address the multi-objective optimization problem of tasks offloading between Cloud and Fog nodes. Table II summarises the various terms used in this paper.

Fog computing represents a service that provides users with an individual networking panel for data processing, rather than relying on centralized Cloud platforms or local IoT devices. It enables users to store, compute, communicate and process data by accessing the entry points of different service providers. This decentralised infrastructure uses nodes on the network for deployment, which are generally more accessible than cloud nodes. Fog nodes are usually cheaper to use but may also be slower.

Tasks offloading is defined as the distribution of task to complete to different nodes in order to optimise certain QoS. Each node and each task has its own parameters, such as memory needed or cpu power. The total price of execution of all the tasks on nodes and the time it takes to perform them depend on these distributions. The goal is to optimize these parameters to efficiently launch an application for users and at a lower cost for service providers.

In mathematical terms, we seek to minimize the total tasks price function $F_1(TVM)$ and the makespan function $F_2(TVM)$ where TVM_{ij} is the binary decision variable which equals to 1 if the task i executes on node j and 0 otherwise. The fitness function therefore is shown in equation 1.

$$min(F_1(TVM_{ij}), F_2(TVM_{ij})) \tag{1}$$

Nodes are defined by four characteristics : performances of the node are determined by the CPU rate in MIPS

Reference	Method	Algorithm	QoS	Type of job	Environment	Tool
1	Deterministic	PTBO	ET, C	IoT tasks	Cloud & Edge	Java
2	Deterministic	Optimal task asignement	ET,C	IoT tasks	Cloud & Edge	GUBORI 9.11
3	Deterministic + blockchain	EESH	E,L,TH,S	IoT tasks	Fog	Java
4	Heuristic	Priotiry based algorithm	RT,C	IoT tasks	Cloud & Fog	CloudSim, CloudAnalyst
5	Heuristic	DPTO	QWT, D	IoT tasks	Cloud & Fog	Non specified
6	Heuristic	Fuzzy logic algorithm	L, EFF	IoT tasks	Cloud & Edge	EdgeCloudSim
7	Ai	DRl	QoE	Mobile device tasks	VEC	Tensorflow
8	Ai	DRL-E2D	M,E	Mobile device tasks	MEC	Tensorflow
9	Ai	FLOR	C, D, L	IoT tasks	6g-V2X	Tianshou
10	Ai	DDMTO	C	Vehicular task	Cloud & Edge	Non specified
11	Ai	DRL+LSTM	CR	Mobile devices tasks	MEC	Non specified
12	Metaheuristic	DNCPSO	C, M	IoT tasks	Ckloud & Edge	WorkflowSim-1.0
13	Metaheuristic	PSOCO	D, C	Mobile device tasks	VEC	Python 3.7
14	Metaheuristic	IWO-CA	E	Generic tasks	Fog	C#
15	Metaheuristic	Marine predator algorithm	M, E	IoT tasks	Cloud & Fog	Java
16	Metaheuristic	Hybrid Genetic Algorithm	RE, QoE, PE	Mobile device tasks	E3C	Non specified
17	Metaheuristic	PGA	E, CT	IoT tasks	Cloud & Fog	Matlab 2018b
Our proposal	Metaheuristic	D-NPGA	C,M	IoT tasks	Cloud & Fog	Python 3.10

TABLE I: Comparison table

F1	Cost function
F2	Makespan function
TVM˙ij	Binary decision variable
cpu_rate˙i	CPU power of node i
cpu_cost˙i	Unit cost of cpu usage for node i
memory_cost˙i	Unit cost of memory usage for node i
bandwidth_cost˙i	Unit cost of bandwidth usage for node i
nb_instruction˙j	Number of instruction for task j
memory˙j	Memory need for task j
input_size_file˙j	Input size file for task j
output_size_file˙j	Output size file for task j
cost˙ij	Cost of executing a task on a certain node
c˙cpu	Cost of using a cpu for completing a task
c˙memory	Cost of using memory for completing a task
c˙bandwidth	Cost of using a bandwidth for completing a task
exec_time˙ij	Time to execute a task on a certain node
min_makespan	Smallest makespan possible
min_cost	Smallest cost possible
fit˙makespan	Objective function for makespan
fit˙cost	Objective function for cost

TABLE II: Table of terms

(Million instruction per second). The cost of doing a task on a specific node is influenced by the CPU cost, memory cost and bandwidth cost. Nodes are created by generating random number using uniform distribution in the interval. We can note that fog and cloud nodes have specific values for each characteristics as they have different advantages and drawbacks in real life.

A task is determined by four other numbers : the number of instruction tells us how big the task is, while the memory required is simply the memory needed to complete the task. Finally, input size file and output size file are the length of data transferred between the user and the node and is used to know how much bandwidth will be used.

In an environment with N nodes and M tasks, we define the Total cost in equation 2.

$$F_1(TVM_{ij}) = \sum_{i=0}^{N}\sum_{j=0}^{M} TVM_{ij} \times cost_{ij} \tag{2}$$

With the cost of executing a task on a certain node :

$$cost_{ij} = c_{cpu} + c_{memory} + c_{bandwidth} \tag{3}$$

$$c_{cpu} = cpu\_cost_i \times \frac{nb\_instruction_j * 1000}{cpu\_rate_i} \tag{4}$$

$$c_{memory} = memory\_cost_i \times memory_j \tag{5}$$

$$c_{bandwidth} = bandwidth\_cost_i \times (input\_size\_file_j + output\_size\_file_j) \tag{6}$$

Secondly, the makespan is defined as the largest execution time of a single node in our solution. Mathematically, it is written as follows:

$$F_2(TVM_{ij}) = max_{i=1->N}(makespan_i) \tag{7}$$

$$makespan_i = \sum_{j=1}^{M} TVM_{ij} \times \frac{nb\_instruction_j * 1000}{cpu\_rate_i} \tag{8}$$

Finally, the values are normalized to have equal influence on the fitness of an individual. We adapted the fitness function presented in [12] for our multi objective setup.

We define the minimum cost and minimum makespan as :

$$min\_makespan = \frac{\sum_{i=1}^{M} nb\_instruction_i * 1000}{\sum_{j=1}^{N} cpu\_rate_j} \tag{9}$$

$$min\_cost = \sum_{i=1}^{M} min_{j=1->N}(cost_{ij}) \tag{10}$$

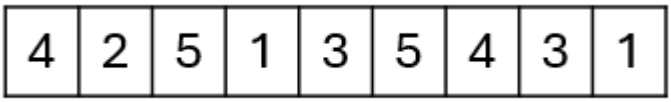

Fig. 2: Chromosome example

The values are divided by the actual cost and makespan of a solution, resulting in a number between 0 and 1. Therefore,our algorithm aims to maximize these two equations:

$$fit_{cost} = \frac{min_{cost}}{F_1(TVM_{ij})} \tag{11}$$

$$fit_{makespan} = \frac{min_{makespan}}{F_2(TVM_{ij})} \tag{12}$$

IV. D-NPGA for task offloading Cloud & Fog

The aim of this work is to determine the most suitable node for executing each task, with the goal of minimizing both the total cost and makespan. To achieve this, we propose a solution called D-NPGA, which is based on NPGA. The general structure of the algorithm is described at 7. Contrary to the majority of state of the art methods that use aggregation formula to deal with multi objective problem, our algorithm takes advantage of the nature of genetic algorithm to generate a set of optimal solution called a Pareto frontier. A Pareto frontier is a list of individual such that none can be better than any other individual in all metrics. This allows the user to choose a solution based on the performances it offers instead of choosing an aggregation variable at the beginning. D-NPGA also differs from classic genetic algorithms due to its unique decision factor. It utilizes niches to select individuals for the next generation. A niche refers to a space in the research area that contains solutions. The greater the number of solutions, the less interesting the individuals in the niche are to our algorithm. A drafting step is added to help edge solution being selected and widen the Pareto frontier. In our program, each individual is represented by a chromosome. It contains multiple genes associated with a single task to execute. If the value of the gene 3 is 7, this means that the task 3 will be executed on the seventh node. This form allows D-NPGA to easily modify individuals and explore the solution space efficiently. 2 shows an example of chromosome.

The first step of the algorithm is to initialize the population. There are many different method to do so but the most common and powerful one is to initialize randomly. Heuristic initialization have been developed but they often provoke premature optimization by reducing the population diversity [14]. Conversely, random initialization creates a lot of diversity that is helpful in finding globally optimal solutions. Therefore, it was the preferred method for D-NPGA. When the first population is created, the genetic process can be applied to it, starting with selection. Here, the population has to be scanned to determine which individual should be used to create the next generation. This is the most crucial step of genetic algorithm as it is generally the only time performances of the solutions are taken into account. If selection has a low evolutive pressure, i.e it selects individuals with low fitness, it will make convergence too long to be used on real time problems. On the opposite side, having high evolutive pressure can lead to premature convergence and worse results as mentioned in [15]. Unlike a classical genetic algorithm that selects only individuals based on their fitness function, our solution uses a tournament to determine which one to choose to create the next generation. The tournament takes place as follows :

- Two individuals are chosen from the initial population. If the individuals are the same, the individuals are drawn again.
- A sub population is chosen randomly from the initial population. The number of individual is determined by a random number between 1 and the length of the population.
- A match is then played between the selected individual. The winner is determined by its dominance over the sub-population. A solution dominates a population if it is always better than any individual in the list on at list one metric. If one of the two individuals dominates and the other not, it wins the match. In the case were both dominates, the first one is declared to be the winner. Finally, if none dominates the sub population, the tournament advances to the next phase. Pseudo code of this step is written in algorithm 1.

Algorithm 1: Match between two individuals

Input: Compared individuals $i1$ and $i2$, Population p
Output: Winning individual or *null*

```
if i1 dominates p then
    return i1;
if i2 dominates p then
    return i2;
return Null;
```

- At this point, a sharing factor is used to determine the winner. The sharing factor is determined with the following formula : $sf_(x_i,P_i) = \sum Sh(1-d(x_i,P_i))$ with $d(x_i,P_i)$ being the distance of an individual x with the rest of the sub-population P on the metric i. The function $sh(a)$ returns 0 when negative and 1 when positive. Finally, we define the shared fitness as $SF(x) = \sum_{i=1}^{n} \frac{f_{ix}}{sf_(x_i,P_i)}$ with f_{ix} the fitness of the individual x on metric i. The individual with the biggest shared fitness is selected to be part of the next generation. In case of a draw, the first one wins the match. This formula allows the algorithm to weight the fitness of the performance of a solution with its "uniqueness". The more a solution is similar to other, the less interesting it becomes. By doing so, genetic diversity

can be preserved, avoiding premature convergence as mentioned in [15]. Pseudo code of this step can be found at 2

Algorithm 2: Shared fitness function

Input: Individual *indiv*, Population p
Output: Shared fitness sf of individual

```
niche_count := [];
for elem in p do
    for i = 0 to indiv.fitness.length do
        sh := 1 − dist(indiv.fitness[i], elem.fitness[i]);
        if sh > 0 then
            niche_count[i] += sh;
sf := 0;
for i = 0 to indiv.fitness.length do
    if niche_count[i] > 0 then
        sf += indiv.fitness[i]/niche_count[i];
return sf;
```

- This step is repeated until the population limit is reached.

Algorithm 3 resumes the general workflow we just described.

Algorithm 3: Selection of new population

Input: Base population p, Number of individual selected p_length
Output: Selected individuals *winners*

```
winners = [];
while winners.length < p_length do
    i1, i2 := Two random individual from population;
    clear_winner := match(i1, i2, population);
    if clear_winner then
        winners.add(clear_winner);
    else
        sf1 := calc_sf(i1, p);
        sf2 := calc_sf(i2, p);
        if sf1 > sf then
            winners.add(i1);
        else
            winners.add(i2);
drafted := drafting(p, winners);
winners.add(drafted);
return winners;
```

In classical NPGA, the population obtained in the last step is used to generate a new one. However, we have added a new step to address an issue with this process. Since the matchmaking is random, it is possible that some individuals will never be selected. While this may be acceptable in general because there are many good solutions in a population, it can be problematic for edge individuals who excel in a single metric. These individuals are typically unique, and losing them means that diversity may never be generated again. To address this issue, we have implemented a draft system. For each individual not selected in the population, their shared fitness is calculated and ranked from best to worst. A selection wheel process is then used to assign a probability based on the ranking. The probability of the nth individual being drafted is $\frac{1}{n}$. The pseudo code of the drafting step can be found at 4.

Algorithm 4: Drafting function

Input: losers,drafting_rate
Output: drafted individuals

```
shared_fitness := [];
l_length := length(losers);
for i := 0 to l_length do
    shared_fitness[i] := calc_sf(losers[i]);
sort(shared_fitness);
a := 1;
drafted_individuals := [];
j := 0;
for i := 0 to l_length do
    rand := random_float(0, 1);
    if drafting_rate/a > rand then
        drafted_individuals[j] := losers[i];
        j := j + 1;
return drafted_individuals
```

After completing the previous step and finalizing the population, the genetic operators are applied. Genetic operators are responsible for the exploration of the solution step after the first initialization of the population. Therefore, they are capital to maintain population diversity and avoid premature convergence. There are three major operators : parent selection, crossover and mutation.

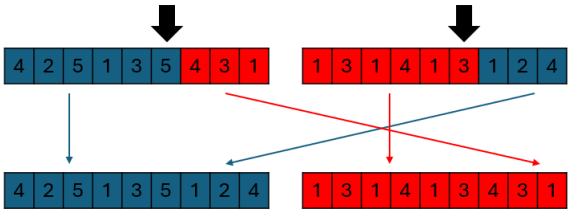

Fig. 3: Single point crossover example

Firstly, two parents need to be selected to generate offspring. A random selection is used as it is the the most popular solution and it gives good results. Secondly, parents are used to generate offspring during crossover. Many crossover exists, with their pros and cons. We decided to use the single point crossover as it is both simple to implement and powerful to explore the solution space. This genetic operator splits the two parents at a random point and combines one part of the first parent's genome with the other part of the second

parent's genome. This process creates two offspring as shown in figure 3. Finally, these new children are mutated to explore the space and potentially discover better genes. Two different mutation types were used: the 'swap' method and the 'value change' method. The 'swap' method involves swapping two gene within an individual as shown in figure 4, while the 'value change' method replaces a gene with another one randomly selected. To determine what will happen to an individual, a first random number is drawn. If it is lower than the mutation rate, a mutation is applied. To choose which one, an integer between one and two is simply generated, one being swap and two value change.

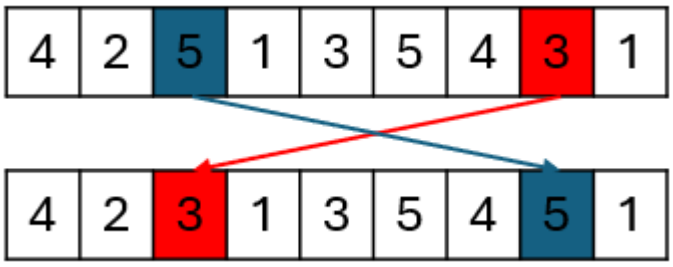

Fig. 4: Swap mutation example

This can be seen in figure 5. The mutated children are then added to the new population, and the parent deleted from the old one. This ensure that all selected individuals will transmit their genes to the next generation. In the case where the number of solution in the population is odd, the last individual is simply added with the mutated children.

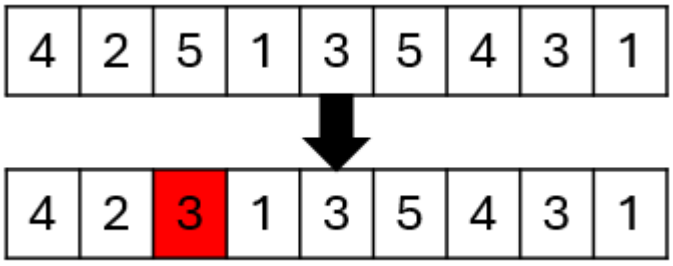

Fig. 5: Value change example

The process will be repeated with the new population until the maximum iteration number is reached.

At the end, the algorithm selects only the individuals that are part of the Pareto set. The flowchart in Figure 6 summarises the steps we just described.

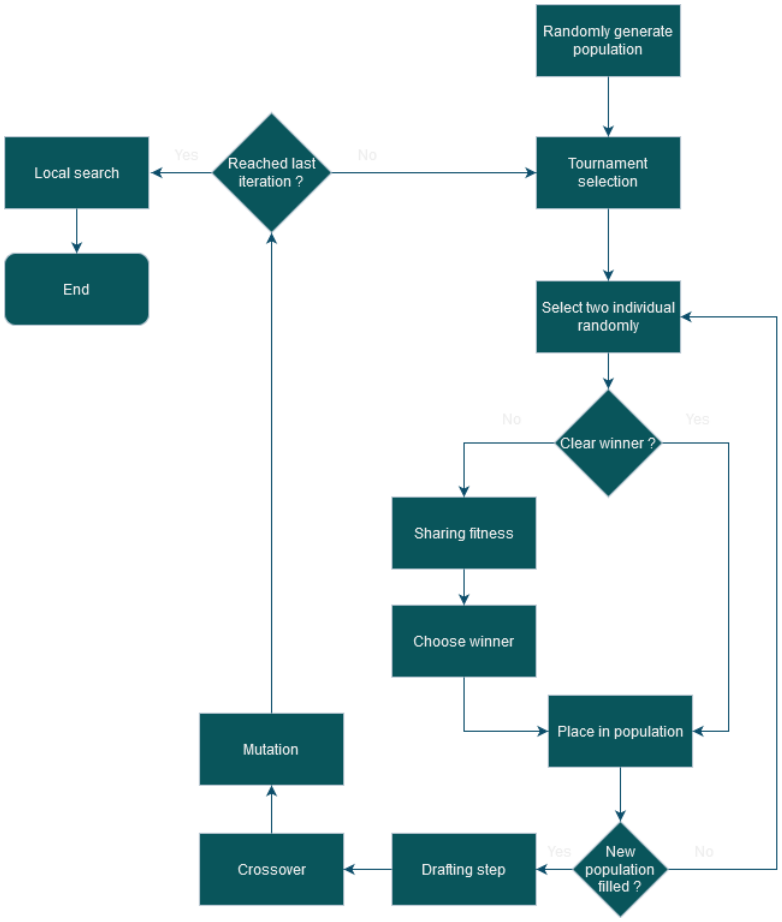

Fig. 6: D-NPGA flowchart

Algorithm 5: D-NPGA

Input: Mutation rate mr, Population length pl, Number of iteration $iter$
Output: Pareto front pf

$pop := initialize\_population(pl)$;
for $i = 0$ *to* $iter$ **do**
  $winners = tournament(population)$;
  $children := crossover(winners)$;
  $population := mutation(children, mr)$;
$pf := pareto\_front(population)$;
return pf;

V. Comparison and results

A. Test environment

Python 3.10.9 was chosen as the programming language for D-NPGA due to its flexibility. The computer configuration used is as follows:

- CPU : Intel(R) Core(TM) i9-9900K CPU @ 3.60GHz
- Memory : 64 Go
- Windows 10 Family

To evaluate our solution, we used the data presented in [12]. As it presents many different approaches and different scales, from tiny problems to huge ones. Three algorithms are implemented in [12], Modified Particle Swarm Optimization (MPSO), Bee Life Algorithm (BLA), and simple Round Robin (RR) to compare with their method, TCaS, a modified version of Genetic Algorithm.
Data is generated using an interval of value for each charac-

teristic. Table III and table IV detail the values used for our tests.

Parameter	Fog nodes	Cloud nodes
CPU rate (MIPS)	[500,1500]	[1000,5000]
CPU usage cost	[0.1,0.4]	[0.7,1.0]
Memory usage cost	[0.01,0.03]	[0.02,0.05]
Bandwidth usage cost	[0.01,0.02]	[0.05,0.1]

TABLE III: Nodes values

Parameter	Value
Number of instructions (10^9 instructions)	[1,100]
Memory required	[50,200]
Input file size	[10,100]
Output size file	[10,100]

TABLE IV: Tasks values

Because our method doesn't give a single solution but rather a Pareto front, we used contribution to compare and evaluate the population generated. It compares the performances of individuals in a population against another group of individual. To do so, it calculates if an individual in population X is dominated or dominates individuals in the population Y. An individual dominates another one if all its QoS are better. If this is true, the other individual is said dominated. The formula of contribution for a population X compared to a population Y is described by the following equation :

$$CONT(X,Y) = \frac{\frac{C}{2} + W_x + N_x}{C + W_x + W_y + N_x + N_y} \tag{13}$$

With :
C the number of solution both in X and Y
W_x and W_y being the number of solution respectively in X and Y dominating some solution in the other population
N_x and N_y being the number of solution respectively in X and Y that aren't dominated by any other individual in the other population

B. Results

The different test we made were done using the same meta parameter as the comparison algorithm we used. Therefore, we selected the same population size (100 individuals) and number of iterations (500) as [12]. Mutation rate will be discussed in the next section.

1) Mutation rate

Mutation rate is one of the key meta parameters of a genetic algorithm. It tells how often a mutation can occur on new individuals. If it is too low, it can limit exploration and prevent the algorithm to find optimal solutions. If it is too high, the algorithm will have a hard time to converge and need longer execution time. Therefore, it is crucial to find the optimal value to limit both these inconveniences. To do so, we ran a test with 20 different mutation rates, from 0 to 0.5 with a 0.025 interval between each. We compared many different criteria such as number of individual in Pareto front, best cost, best makespan. We also used contribution by comparing the output of a D-NPGA run with a certain mutation rate to all the other runs and calculated the mean. To have general results, we did this comparison on 30 different datasets.

Table V shows the results of theses tests. We can first notice that a mutation rate of 0 is the worst case. Without mutation, it is impossible to generate new genes. Genes generated randomly at the start are very unlikely to create globally optimal solutions and can only create locally optimal individual. This explains why the first result is so far behind the other. By analysing the rest of the table, multiple trends can be observed. In general, the algorithm ran with a mutation rate of 0.2 seems to have the best results. It produces more Pareto solution than any other, produces the best price and has one of the best contribution. However, the makespan produced is not one of the best. Indeed, makespan is lower when few to no nodes are used multiple times and high when a node is highly exploited. Therefore, it benefits from high exploration and low selection. So, by increasing the mutation rate, D-NPGA optimizes makespan to the detriment of cost, which benefits from exploiting profitable nodes. The best balance between those two is reached when the number of individual generated is the higher, which is when the mutation rate is 0.2. This is therefore the one we will choose.

2) Local search

Another important key parameter of D-NPGA is the number of iteration of the local search. Because it is used at the end of the algorithm, we don't need to worry about local convergence, which is one of the biggest weakness of local search. But this method takes some time to be executed and using it on a population of 100 individuals may slow down significantly D-NPGA. Therefore, the number of iteration needs to be keped low enough not to impact execution speed, and high enough that it has a significant impact on the population quality and diversity. To determine this optimum, a local search was used on a population generated and optimized by D-NPGA over 500 iterations. Figure 7 shows the result of this test. The number of swap is obviously the highest during the first few iteration. This value slowly decreases to around 10 swap after the 150th iteration. This correspond to a fifteenth of the execution time of D-NPGA, which is acceptable.

To test the effectiveness of this threshold, we plotted the Pareto front of D-NPGA before and after the local search. Figure 8 shows that the added step had a significant impact on both the performances and the diversity of the Pareto front.

3) Convergence

The convergence was analysed to determine the duration required for our method to identify optimal solutions. The algorithm was executed with 80 tasks, and the best individual with the best cost and the individual with the best makespan were tracked at each iteration. Figures 9 and 10 illustrate the evolution of these values over 500 iterations. The makespan reached its convergence point at approximately 100 iterations, while the cost appeared more erratic and reached its minimum at 500 iterations.

Mutation rate	Number of solution	Best cost	Best makespan	Contribution
0	10.47	1319.39	290.26	0.05
0.1	23.43	1221.68	246.03	0.87
0.2	26.97	1208.66	216.13	0.82
0.3	25.67	1210.68	202.97	0.83
0.4	24.87	1221.39	199.55	0.69
0.5	23.07	1231.72	197.26	0.62
0.6	23.57	1242.98	197.32	0.54
0.7	21.20	1264.01	197.84	0.40
0.8	20.23	1268.46	197.31	0.34
0.9	18.67	1279.29	197.65	0.26

TABLE V: Mutation rate tests results

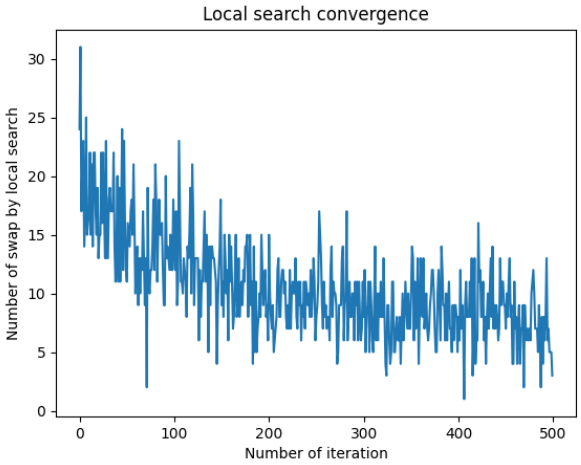

Fig. 7: Number of swap by local search over 500 iterations

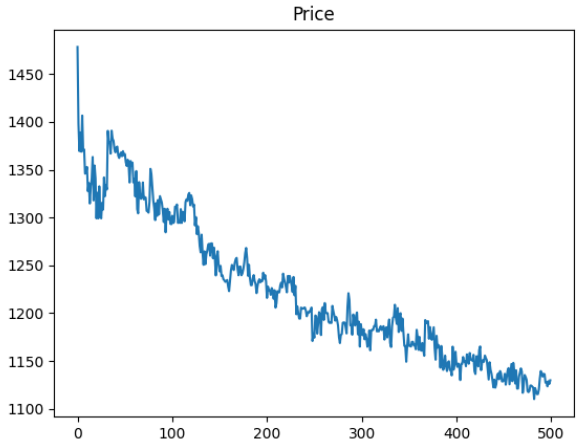

Fig. 9: Price convergence

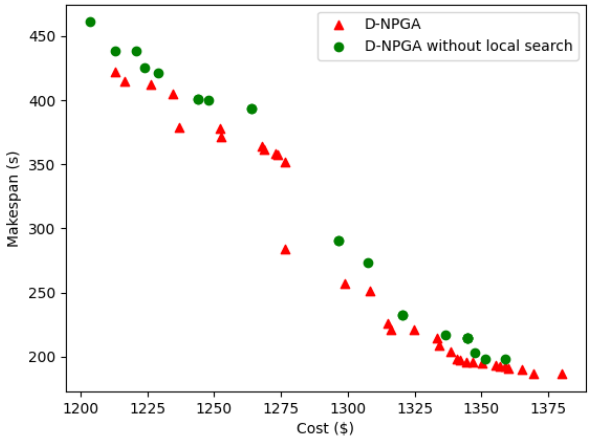

Fig. 8: Pareto front before and after local search

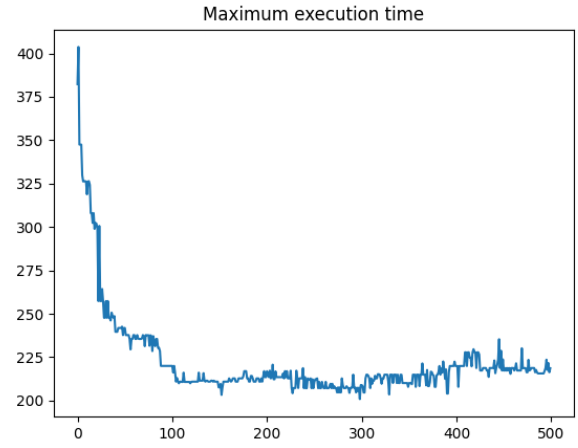

Fig. 10: Makespan convergence

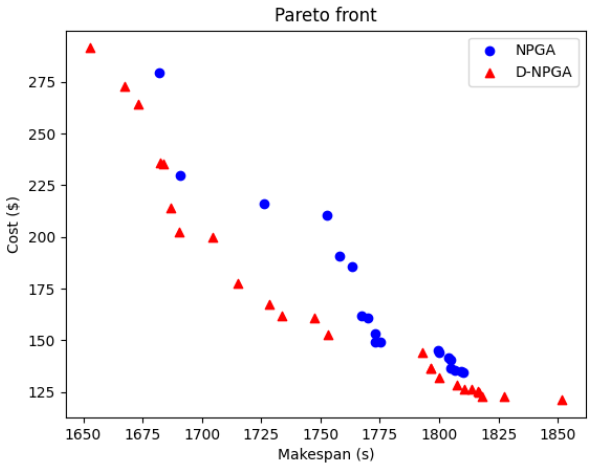

Fig. 11: Pareto set of D-NPGA

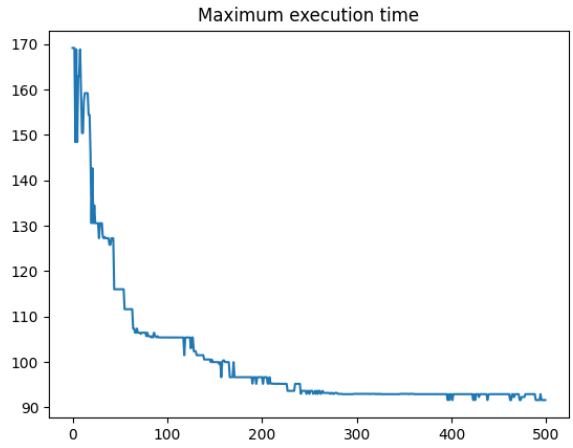

Fig. 12: Convergence of makespan for 65 node

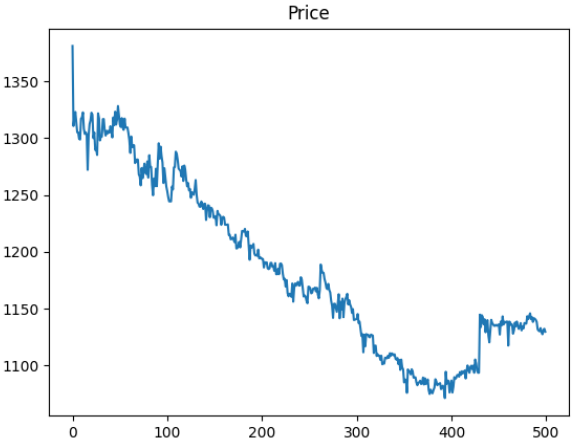

Fig. 13: Convergence of price for 65 node

4) Population diversity

To visualise the distribution of optimal solutions, the Pareto set is plotted after the final iteration. Figure 11 displays the set for D-NPGA and NPGA on the same starting population. We can see that our method proposes a greater number in extreme cases. This demonstrates that D-NPGA offers greater diversity than the original algorithm.

Another indicator of diversity of a population is the number of Pareto solution. The more solution there are, the better it is for the user that can choose the individual that correspond precisely to the QoS he aims for. Figure 11 showed that D-NPGA produces more individual than classical NPGA, but it needed to be tested on multiple dataset. Therefore, we generated 30 different datasets and compared the number of Pareto solutions for both D-NPGA and NPGA. Our method generated on average 25 solution while the former one only 15. This means that D-NPGA produce 66% more Pareto individuals than the base algorithm.

Both these tests indicate indeed that our method proposes a varied Pareto front more varied than what is produced by classical algorithms.

5) Number of node

To test the scaling of D-NPGA, the number of node and task was indeed to observe the evolution of the QoS. The increase number of task will be discussed in the comparison section. For the number of node, we tested 6 different values : 10/3 (10 fog node and 3 cloud node), 12/5, 15/6, 20/8, 30/10 and 50/15. The same four factors calculated for mutation rate were used here. Table VI shows the results obtained.

As we can expect, the more node there are, the better the individuals produced. Between the smallest and biggest dataset, the price has been reduced by 8% and the makespan by 38%. The contribution also increased with these value. We can note that there is a clear difference between the upgrade in cost and makespan. As the number of node increase, the more unlikely it becomes for a node to be used multiple time. Indeed, mutations are much more likely to add a new gene or one that was slightly used than one that was highly exploited. In addition, selection doesn't have enough iteration to select nodes that are profitable to find globally optimal individual in cost. Therefore, makespan highly benefits from the increasing number of node while cost does only slightly. To confirm this hypothesis, we can look at the convergence of cost and makespan for the 50/15 dataset.

Figure 12 shows indeed that makespan reaches a global minimum at 250 iteration while price obtained its best results after 400 iterations. We can see that, while it prefers makespan, D-NPGA can still produce excellent results for both metrics with a much bigger dataset and without adjusting the number of iteration, proving its capacity to scale well with the number of data.

6) Load balancing

Load balancing is an important feature of Fog Cloud environment. It tracks the number of nodes used that are in the fog and the number that are in the cloud. This is an important

Number of node	Avg number of Solutions	Avg best Cost ($)	Avg best Makespan (s)	Avg contribution
10 fog 3 cloud	26	1245.34	206.80	0.20
12 fog 5 cloud	22	1264.25	145.91	0.23
15 fog 6 cloud	20	1278.99	126.96	0.31
20 fog 8 cloud	18	1252.05	98.23	0.35
30 fog 10 cloud	14	1252.18	93.03	0.42
50 fog 15 cloud	13	1152.58	79.49	0.44

TABLE VI: Node scaling tests results

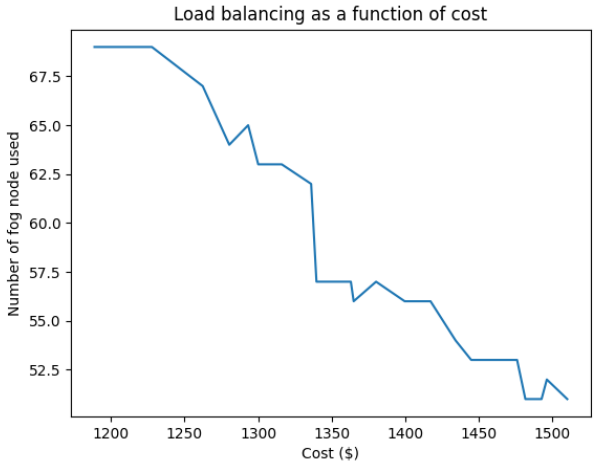

Fig. 14: Load balancing as a function of cost

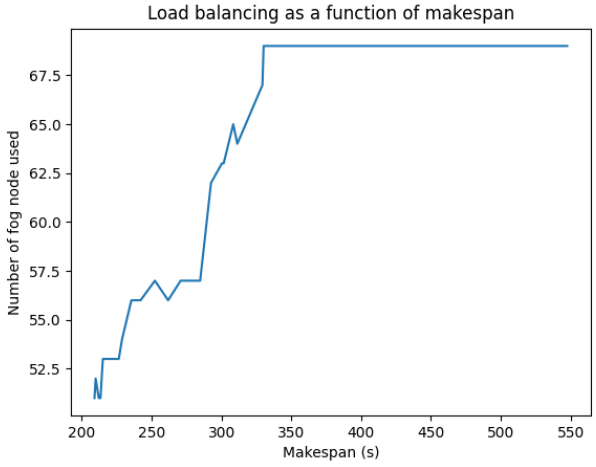

Fig. 15: Load balancing as a function of makespan

metric as

C. Comparison

To match the experimental setup of the data we use, we chose the same number of tasks (40, 80, 120, 200, 300 and 500) and for each step, we ran our algorithm 30 times with new dataset generated each time. As our method does not output a unique solution, we had to select an individual for comparison with the other algorithms mentioned in [12]. After conducting numerous tests, we concluded that the best solution was the one with the smallest makespan, as it consistently aligned with the values provided by the other algorithms.

The following table shows the obtained results :

Number of tasks	TCaS	BLA	MPSO	RR	D-NPGA
40	**114.37**	185.37	157.41	313.01	128.60
80	232.91	368.78	352.50	965.32	**223.29**
120	357.42	580.29	546.70	1267.54	**345.78**
200	633.28	894.42	873.06	1521.44	**600.36**
300	1052.92	1436.58	1375.54	2861.95	**962.31**
500	1994.21	2512.40	2425.86	4633.48	**1695.38**

TABLE VII: Makespan results

We can see that our solution is better by a large margin than all the others in terms of cost. A 15% cost optimization was achieved across all task numbers, which remained consistent regardless of the dataset scale. It should be noted that D-NPGA can be outperformed for a small number of task for makespan. In cases where the dataset scale is 40, TCaS proposes a better solution for this QoS. Our solution results in a 12% increase in makespan for the smallest number of tasks, but outperforms by 4% for 80 tasks. At 500 tasks, our solution is superior in both cost and time by 15%. In contrast, the other algorithms perform worse than our solution at every scale and for all metrics.

The proposed algorithm has two major advantages. Firstly, it outperforms the state of the art when the problem size scales up. In real-life scenarios, where the number of tasks to offload is generally high, it is more important to perform well at larger scales. Secondly, the comparison was made using

Number of tasks	TCaS	BLA	MPSO	RR	D-NPGA
40	861.53	811.34	811.45	859.96	**685.52**
80	1817.90	1732.46	1689.26	1793.53	**1426.04**
120	2779.72	2639.79	2583.55	2687.34	**2192.95**
200	4403.63	4247.69	4134.09	4323.92	**3619.09**
300	6740.36	6591.75	6405.11	6666.58	**5406.06**
500	11,299.53	11,219.18	10,823.84	11,266.31	**9412.73**

TABLE VIII: Total cost results

only one individual. Actually, our algorithm generated a set of near Pareto optimal solutions. This approach allows for the exploration of multiple trade-offs simultaneously, enabling the selection of the most optimised solution.

VI. Conclusion

With the rapid growth of wireless communication and the development of new paradigms like fog computing, it is necessary to optimize as much as possible makespan and cost when trying to offload tasks. Our new solution, D-NPGA, not only provides better results than the state of the art and scales better, but also generates many near Pareto optimal solutions that explore different trade-offs simultaneously and propose extreme solutions better than the original algorithm. Our future work will focus on improving the execution time of D-NPGA and testing the impact of increasing the number of nodes and tasks even further.

References

[1] Future of industry ecosystems: Shared data and insights. https://blogs.idc.com/2021/01/06/future-of-industry-ecosystems-shared-data-and-insights/. Accessed : 28/02/2024.

[2] Qu'est ce que le fog computing ? https://iotindustriel.com/glossaire-iiot/fog-computing/. Accessed : 28/02/2024.

[3] State of iot 2023: Number of connected iot devices growing 16 https://iot-analytics.com/number-connected-iot-devices/. Accessed : 28/02/2024.

[4] Mohamed Abdel-Basset, Reda Mohamed, Mohamed Elhoseny, Ali Kashif Bashir, Alireza Jolfaei, and Neeraj Kumar. Energy-aware marine predators algorithm for task scheduling in iot-based fog computing applications. *IEEE Transactions on Industrial Informatics*, 17(7):5068–5076, 2021.

[5] Mainak Adhikari, Mithun Mukherjee, and Satish Narayana Srirama. Dpto: A deadline and priority-aware task offloading in fog computing framework leveraging multilevel feedback queueing. *IEEE Internet of Things Journal*, 7(7):5773–5782, 2020.

[6] Tejaswini Choudhari, Melody Moh, and Teng-Sheng Moh. Prioritized task scheduling in fog computing. In *Proceedings of the ACMSE 2018 Conference*, ACMSE '18, New York, NY, USA, 2018. Association for Computing Machinery.

[7] J. Horn, N. Nafpliotis, and D.E. Goldberg. A niched pareto genetic algorithm for multiobjective optimization. In *Proceedings of the First IEEE Conference on Evolutionary Computation. IEEE World Congress on Computational Intelligence*, pages 82–87 vol.1, 1994.

[8] Farooq Hoseiny, Sadoon Azizi, Mohammad Shojafar, Fardin Ahmadizar, and Rahim Tafazolli. Pga: A priority-aware genetic algorithm for task scheduling in heterogeneous fog-cloud computing. 05 2021.

[9] Pejman Hosseinioun, Maryam Kheirabadi, Seyed Reza Kamel Tabbakh, and Reza Ghaemi. A new energy-aware tasks scheduling approach in fog computing using hybrid meta-heuristic algorithm. *Journal of Parallel and Distributed Computing*, 143:88–96, 2020.

[10] Zhongjin Li, Victor Chang, Jidong Ge, Linxuan Pan, Haiyang Hu, and Binbin Huang. Energy-aware task offloading with deadline constraint in mobile edge computing. *EURASIP Journal on Wireless Communications and Networking*, 2021, 03 2021.

[11] Quyuan Luo, Changle Li, Tom H. Luan, and Weisong Shi. Minimizing the delay and cost of computation offloading for vehicular edge computing. *IEEE Transactions on Services Computing*, 15(5):2897–2909, 2022.

[12] Binh Minh Nguyen, Huynh Thi Thanh Binh, Tran The Anh, and Do Bao Son. Evolutionary algorithms to optimize task scheduling problem for the iot based bag-of-tasks application in cloud–fog computing environment. *Applied Sciences*, 9(9), 2019.

[13] Zhaolong Ning, Peiran Dong, Xiaojie Wang, Joel J. P. C. Rodrigues, and Feng Xia. Deep reinforcement learning for vehicular edge computing: An intelligent offloading system. *ACM Trans. Intell. Syst. Technol.*, 10(6), oct 2019.

[14] Eneko Osaba, Roberto Carballedo, Fernando Díaz, Enrique Onieva, Pedro López García, and Asier Perallos. On the influence of using initialization functions on genetic algorithms solving combinatorial optimization problems: A first study on the tsp. 06 2014.

[15] Hari Mohan Pandey, Ankit Chaudhary, and Deepti Mehrotra. A comparative review of approaches to prevent premature convergence in ga. *Applied Soft Computing*, 24:1047–1077, 2014.

[16] Shubhkirti Sharma and Vijay Chahar. A comprehensive review on multi-objective optimization techniques: Past, present and future. *Archives of Computational Methods in Engineering*, 29:3, 07 2022.

[17] Jung-Fa Tsai, Chun-Hua Huang, and Ming-Hua Lin. An optimal task assignment strategy in cloud-fog computing environment. *Applied Sciences*, 11(4), 2021.

[18] Xiaohui Zhao, Liqiang Zhao, and Kai Liang. An energy consumption oriented offloading algorithm for fog computing. pages 293–301, 08 2017.